



KTU NOTES APP

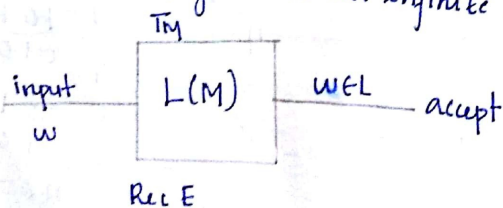


www.ktunotes.in

20/10/2017 6. Recursive & Recursively Enumerable Language

~~Recursive~~ ~~Enumerable~~ ~~Language~~ ~~L~~ is

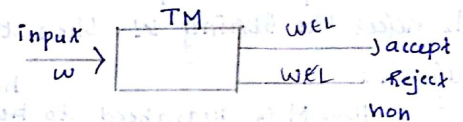
A language L is said to be recursively enumerable if there exist a TM that accepts it, i.e., there exist a Turing machine M such that for every $w \in L$, $q_0 w \xrightarrow{*} x_1 q_f x_2$ with q_f is a final state and x_1 and x_2 are strings after processing w . This definition says nothing about what happens for w not an element of L . It may be the machine halts in a non-final state or it never halts and goes into an infinite loop.



Recursive Language:

A language L is called recursive if for some Turing machine M such that,

- If $w \in L$ then M accepts w and halts.
- If $w \notin L$ then M rejects w and halts on non-final state.



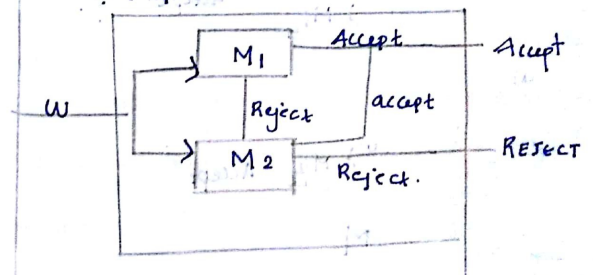
31/10/2017 Closure properties of Recursive and Recursively Enumerable language

Property 1

The union of two recursive languages are recursive. [recursive language is closed under union]

PROOF

Let L_1

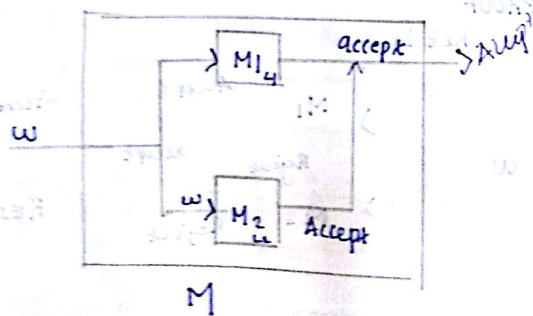


Let L_1 be a recursive language accepted by M_1 . L_2 be a recursive language accepted by M_2 . We construct a new TM that accepts $L_1 \cup L_2$. If w is accepted by M_1 then M accept. If M_1 reject w then the m/c M stimulates M_2 and accepts if M_2 accept. Since both M_1 and M_2 reject the string w then the m/c M reject w .

Here M is guaranteed to halt on all inputs.

Property 2

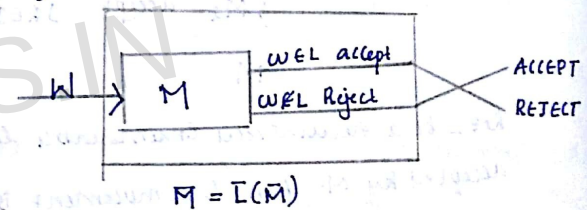
Union of two recursively enumerable language (recursively enumerable language are closed under union)



Let L_1 be a recursively enumerable language accepted by M_1 and L_2 be a recursively enumerable language accepted by M_2 . We construct a new TM M such that it accepts the language $L_1 \cup L_2$ is shown in above figure.

Property - 3

If L is recursive language then complement of L i.e. L' is also recursive [Recursive languages are closed under complementation.]



11/11/2017

Let L be a recursive language accepted by TM M and L' is complement of L .

" " $\bar{M}$

TM M modifies as follows to create $\bar{M}$

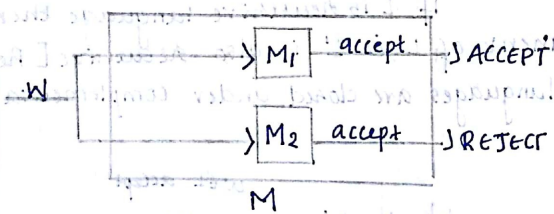
- * Accepting states of M are made non accepting states of $\bar{M}$
- * All non accepting states of M are made

acceptin state of M

* Since M guaranteed to halt on all i/p's w so, $\bar{M}$ $\therefore \bar{M}$ is also a recursive language.

Property-4

If both language L and its complement are recursively enumerable then L is recursive.

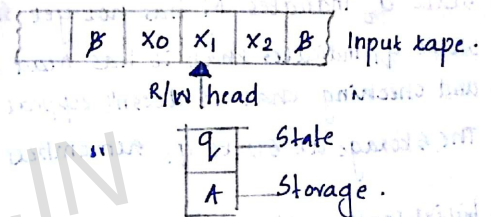


Let L be a recursively enumerable language accepted by M . Then L complement is the complement of L accepted by M . M_2 we create a new TM M using M_1 and M_2 shown in above figure. From above figure it is guaranteed that M halt on all inputs of w . $\therefore$ the language accepted by M is L is recursive.

Programming Techniques In TM

1) Storage or State / finite Control

In this type of TM states in the control unit not only to represent current position of the Prgm but also to hold a finite amount of data. Below figure suggest this technique.



A state in TM is a tuple $[q, A]$ where q is a state and A is the stored value at state q . The transition function δ can be viewed as follows

$$\delta([q, A], X) = ([p, B], Y, D)$$

↓

Current State
+
Stored Value

↓

Input

↓

next State
+
Stored Value

↓

repla
+
cing
Symbol

↓

direction

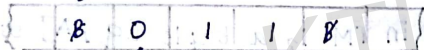
Q: Design a TM that accept the string generated by the language $10^* + 01^*$ using storage at the state concept

Soln: Let M be a TM corresponds to the language $10^* + 01^*$

The state in the Q remembers what the TM is doing.

- * State q_0 indicates M has not yet read its first symbol
- * State q_1 indicates that it has read the first symbol and checking that it doesn't appear else where.
- * The storage at state q_1 remembers the first symbol read.

Initial Configuration:



$$\delta([q_0, B], 0) = ([q_1, 0], 0, R)$$

$$\delta([q_1, 0], 1) = ([q_1, 0], 1, R)$$

$$\delta([q_1, 0], B) = ([q_2, B], B, H)$$

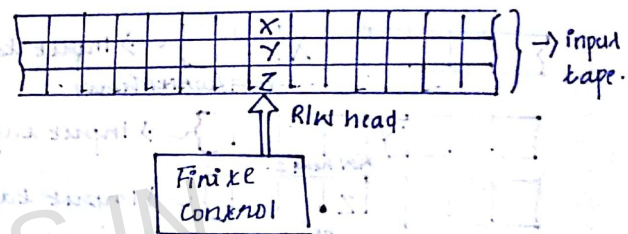


$$\delta([q_0, B], 1) = ([q_1, 1], 1, R)$$

$$\delta([q_1, 1], 0) = ([q_1, 1], 0, R)$$

$$\delta([q_1, 1], B) = ([q_2, B], B, H)$$

2) TM with multiple tracks on single Tape.

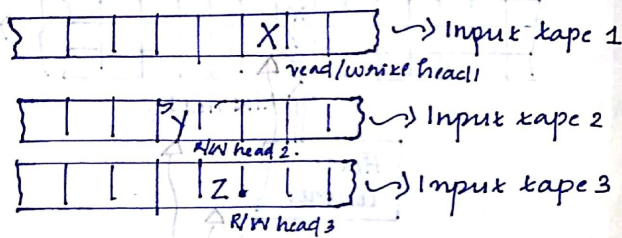


In this technique the i/p tape of the TM is divided into several tracks each track can hold all. Each track are divided into several cells. Each cell can hold one symbol. The Tape alphabet of TM consists of tuples with one component for each track below fig illustrates TM with multiple tracks on a single input tape, A delta function(s) on a multitrack TM can be represented as

$$\delta(q, [x, y, z]) = (p, [A, B, C], D)$$

$\downarrow$ current state $\downarrow$ input symbol $\downarrow$ next state $\downarrow$ replacing symbols $\downarrow$ direction of head movement (R/L)

3) Multitape TM



The device has finite control unit and multiple no. of input tapes. Each tape is divided into cells and each cell holds one symbol of the finite tape alphabet (Σ). The above fig illustrates multitape TM with three i/p tape. The transition function of multiple tape TM depends on current state and the symbol scanned by each of the tape head.

In 3-Tape TM δ can be represented as,

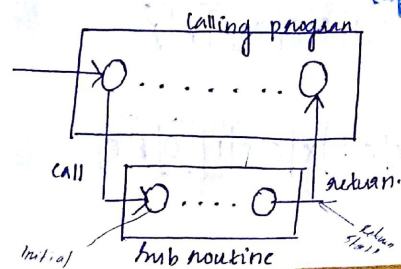
$$\delta(q, [x, y, z]) = (p, [A, D_1], [B, D_2], [C, D_3])$$

$\downarrow$ current state $\downarrow$ i/p symbol scanned by R/W head 1/2/3 respectively $\downarrow$ next state $\downarrow$ direction of head movement (Right/left)

A, B, C — replacing symbol

- In one move, a multi-tape TM does the following
 - The control enters into a new state (p)
 - On each tape, a new symbol is written on the cell being scanned
 - Each head makes a move which can be written right or left

Sub routine.



Copy Subroutine TM

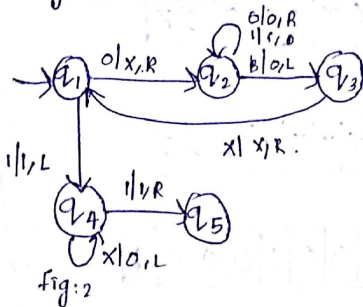


Fig: 2

The heart of this algorithm is a subroutine we call it has a copy. This subroutine algorithm copies the block of n zeros to the right end of input tape. Fig. 1 shows multiplier TM and fig. 2 shows copy subroutine TM.

Tape status after 1st iteration (first subroutine call).

{ B | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | B | B }

Final output (after n th iteration)

0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0

Enumerating TM.

A TM can be identified with integer. A TM $M = (Q, \Sigma, \Gamma, \delta, B, q_0, F)$ can be represented as a binary strings. For that we must assign different integers to the state, tape symbols & direction, each transition function in enumerated TM can be represented by $\delta(q_i, X_j) = (q_k, X, D_m)$ then code for each function is

0ⁱ10^j10^k10^l10^m

The complete code for TM M consists of set of transmission code separated by pair of ones.

$c_1 || c_2 || c_3 || \dots || c_n$

eg: Suppose the TM M equal to $M = (Q, \Sigma, \Gamma, \delta, B, q_0, F)$

$Q = \{q_1, q_2, q_3\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, B\}$

$\delta(q_1, 1) = (q_3, 0, R)$

$\delta(q_3, 0) = (q_1, 1, R)$

$\delta(q_3, 1) = (q_2, 0, R)$

To enumerate above TM first we must assign different integers (or binary strings) to the states, tape symbol & direction.

Enumerating Symbols	Tape Symbols	Direction
$q_1 = 0$	$0 = 0$	$L = 0$
$q_2 = 00$	$1 = 00$	$R = 00$
$q_3 = 000$	$B = 000$	

Code for first transition function

$$\delta(q_1, 1) = (q_3, 0, R)$$

$$c_1 = 0100100010100$$

Code for 2nd transition function

$$\delta(q_3, 0) = (q_1, 1, R)$$

$$c_2 = 0001010100100$$

Code for 3rd transition function

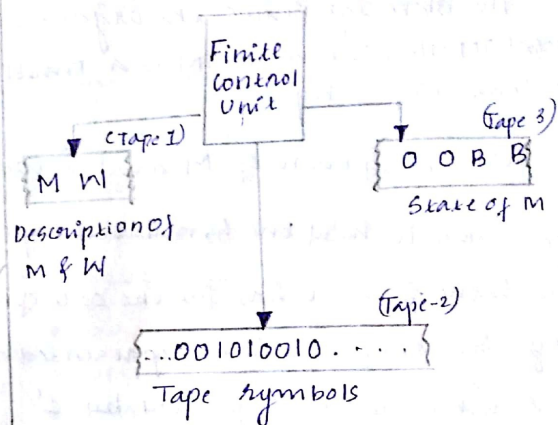
$$\delta(q_3, 1) = (q_2, 10, R)$$

$$c_3 = 00010010010100$$

$$TM = c_1 || c_2 || c_3 ||$$

$$0100100010100 || 0001010100100 || 00010010010100 ||$$

UNIVERSAL TM



A TM is a special purpose computer. Once it is designed the m/c is restricted to carry out one particular type of computation. A reprogrammable TM called universal TM (M_u) is an automaton that given as input the description of any TM M and a string w . M_u can simulate the computation of M on w .



Organisation of a Universal TM

- A universal TM is a multi tape TM

The above fig shows the organisation of universal TM. Here universal TM is a multi tape TM consist of 3 tapes

- * First tape holds description of M and input w .
- * Second tape is used to hold the simulated tape of M . Using the same format has for the code of M . i.e. tape symbol x_i of M will be represented as 1^i . The tape symbols will be separated by '1'.

- * Third tape of M_u hold the current state of M . In which the state q_i represented by i no. of zeros.

The operatⁿ of M_u can be summarised as follows

Step 1: Check code for M is valid for some TM ' M '. If not, halt without accepting

Step 2: Initialise second tape to contain the input w in its coded form.

eg:-

q_p	0	1	β
code format	0	00	000

Step 3: Place start state of M to the third tape. Then move M_u 's head of second tape to the 1st simulated cell.

Step 4: To simulate a move of M , M_u searches on its 1st tape for a transition $0^i | 0^j | 0^k | 0^l | 0^m$ such that 0^i in the state on tape 3. 0^j is the tape symbol of M that begins at position $\bullet$ on tape 3.

This transition is:

- changes the content of tape 3 to 0^k +
- Replace 0^j with 0^l on tape 2
- Move head on tape 2 to the left/right depending on the value of 0^m

Step 5: If M has no transition that matches simulated symbol and tape — in step 4. M halt in the simulated configuration M .

6. If M enters its accepting state then M_u accept.

→ Non-Deterministic TM (NTM)

A non-deterministic TM (NTM) differs from the deterministic TM (DTM) by having a transition function δ such that for each state q and tape symbol x , $\delta(q, x)$ is a set of ~~any~~ tuples.

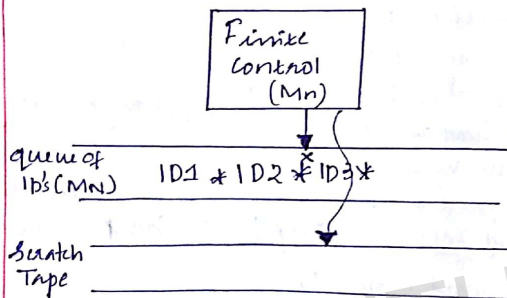
$$\delta(q, x) = \{(q_1, y_1, d_1), (q_2, y_2, d_2), \dots, (q_k, y_k, d_k)\}$$

where k is any +ve integer

Theorem

Equivalence of DTM & NTM

If M_N is a NTM, then there is a DTM M_D such that $L(M_N) = L(M_D)$
 M_D is designed multitape TM shown in below figure.



The first tape of M_D holds a sequence of IDs of M_N , including the state of M_N & ID of M_N is marked as current ID, whose successor IDs are in the process of being discovered. All IDs to the left of the current one have been explored and can be ignored subsequently.

The process the current ID M_D does the following

1. M_D examines/checks the state and scan symbol of the current ID. If the state in the current ID is accepting then M_D accepts and simulates M_N .
2. However if the state is not accepting then state-symbol combination has k moves. Then M_D uses its second tape to copy the ID.
3. M_D modifies each of k ID according to a different one of the k -choices of moves that M_N has from its current ID.
4. M_D returns to the marked current ID, erases the mark and moves the mark to the next ID to the right. The cycle then repeats with step 1.

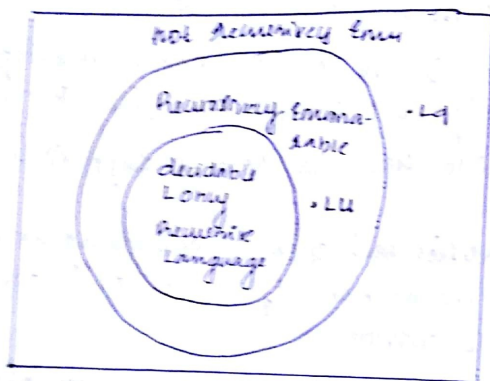
3%

Decidability

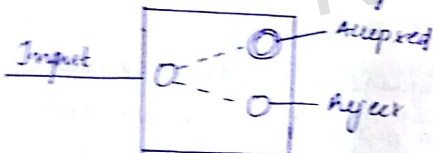
A language L is called decidable if there is a TM which accepts and halts on every IP string.

- Every decidable languages are recursive languages.
- Every " " " Turing acceptable
- Recursive languages are corresponds to algorithm & a well defined sequence of steps to solve a problem.

Below figure shows relationship b/w recursively and non recursively enumerable languages.



For a decidable language, for each i/p string the TM halt either at accept or the reject state as depicted in the following diagram.



→ Eg's of decidable language

Is a number 'P' prime

Given a regular language L and string w

If $w \in L$.

- Does a given DFA accept the empty language?

Undecidable language

A language L is called Undecidable if it is not a recursive language.

- A decision problem 'P' is called undecidable if the language L of all 'Yes' instances to P is not decidable.
- Undecidable language are not recursively enumerable but sometimes there may be semi-decidable.
- non recursively enumerable lang. corresponds to Prouce.
- Example
- The halting problem of TM
- Post Correspondence Problem (PCP)
- clique problem
- Vertex cover problem.
- 3 CNF problem.

⇒ TM halting Problem

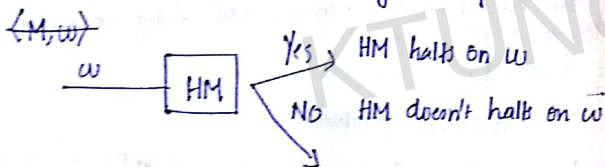
Input to halting Problem: $\langle M, w \rangle$, a TM M and i/p string w

problem definition: does a TM finish computing of the string w in a finite no. of states/steps?
The answer must be either Yes or No

PROOF:

At first, we will assume that such TM problem exist to solve this problem we call it as Halting Machine (HM) That produces 'Yes' or 'No' in a finite amount of time. If the halting rule in a finite " " " " the o/p. comes Yes otherwise No

The following is the block diagram of HM

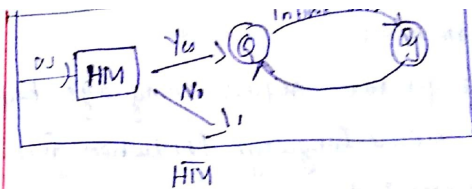


Now we will define an inverted halting ^{n/s} HM as follows,

* If HM returns Yes then HM loops forever

* If HM returns No then HM halts.

The following fig is block diagram of HM.



Further a machine HM_2 which i/p itself is constructed as follows. If HM_2 halts on i/p loop forever else halt.

Here we got a contradiction Hence the halting problem is undecidable.

Chomsky Classification of Grammars.

According to Noam chomsky there are four types of grammars they are type 0, type-1, type-2 and type-3. The following table shows how they are differ from each other.

Grammar type	Grammar Accepted	Language Accepted	Automation
TYPE 0	Unrestricted Grammar	Recursively Enumerable language	Turing Machine (TM)
TYPE 1	Context sensitive Grammar	Context sensitive language	Linear bounded Automata (LBA)
TYPE 2	Context free Grammar	Context free language	Push down Automata (PDA)
TYPE 3	Regular Grammar	Regular language	Finite State Automaton (FSA)

TYPE-3 GRAMMAR:

It generates regular language. Regular grammars are linear languages. Production format of linear grammar is defined as follows:

$$X \rightarrow a$$

$$X \rightarrow aX \text{ where } X \in V \text{ and } a \in T$$

eg: Let $G = (V, T, S, P)$ be a regular grammar then $P = \left\{ \begin{array}{l} S \rightarrow a \\ S \rightarrow aS \end{array} \right\}$

Type-3 language are accepted by finite state automaton.

TYPE-2 Grammar

It generate context free language. It is accepted by pushdown automaton. Type-2 grammar or context free grammar is defined as, $G = \{V, T, S, P\}$ where all productions are of the form $V \rightarrow (VUT)^*$ where $V \in V$ is a variable and T is a terminal.

eg: $L = a^n b^n$ where $n \geq 1$

$$\text{Let CFG, } G = \{V, T, S, P\}$$

$$P = \left\{ \begin{array}{l} S \rightarrow aSb \\ S \rightarrow ab \end{array} \right\}$$

Type-1 Grammar

Type-1 grammar generate context sensitive language. It is accepted by Linear bounded automaton. Type-1 grammar is defined as $G = \{V, T, S, P\}$ where all productions are of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$ where $A \in V$ and $\alpha, \beta, \gamma \in (V \cup T)^*$

NOTE:

A string α, γ, β may be empty but A must be non-empty.

* The rule $\alpha \rightarrow \epsilon$ is allowed if α doesn't appear on the RHS of any rule.

eg: $G = \{V, T, S, P\}$ $P = \left\{ \begin{array}{l} S \rightarrow aBac \\ Ba \rightarrow cDcD \end{array} \right\}$

TYPE-0 Grammar:

It generates recursively enumerable language. It is accepted by Turing Machine. $G = \{V, T, S, P\}$ be a type-0 grammar. Then all productions gives $\alpha \rightarrow \beta$ where α contains strings of terminals and variable with atleast one variable on α side.

Relationship b/w Type-0, 1, 2, 3 gram

